

PYTHON WEEK 2

Data Structures: Collections & Mappings

MIT KATHMANDU — ACCELERATED COURSE

| Session 1: ATM Project Review

Key Takeaways from Week 1:

- ✓ Logic flow with while True
- ✓ State management using variables
- ✓ User input validation and casting



| Why Data Structures Matter



Organization

Isolated variables (x, y, z) are hard to manage. Collections keep related data together.



Efficiency

Specific structures are optimized for searching, sorting, or unique data storage.



Readability

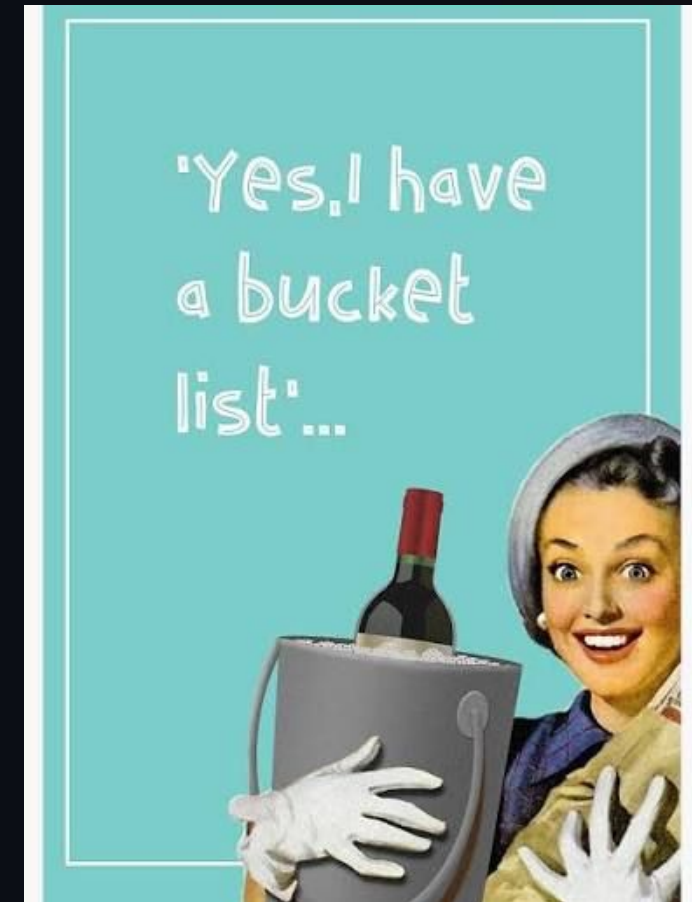
Programs become more intuitive when you use the "right tool for the job."

| Lists: Mutable Collections

Lists are **ordered**, **mutable** sequences of items.

```
# List Syntax
fruits = ["apple", "banana", "cherry"]
mixed = [1, "Hello", 3.14, True]
```

- Allows duplicate items
- Elements can be any type
- Can grow or shrink dynamically



| Zero-Based Indexing

Item	"P"	"y"	"t"	"h"	"o"	"n"
Positive Index	0	1	2	3	4	5
Negative Index	-6	-5	-4	-3	-2	-1

```
langs = ["Python", "C++", "Java", "Ruby"]  
print(langs[0])    # Output: Python  
print(langs[-1])   # Output: Ruby (The last item)
```

| List Slicing: [start:stop:step]

Extract a sub-portion of a list. The stop index is **exclusive**.

```
nums = [0, 10, 20, 30, 40, 50]

# Items from index 1 to 3
print(nums[1:4]) # [10, 20, 30]

# First three items
print(nums[:3])  # [0, 10, 20]
```

The Python List Slice Technique



| Advanced Slicing: Step

The full syntax is `list[start:stop:step]`.

```
vals = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Every second item
print(vals[::2])      # [0, 2, 4, 6, 8]

# Reverse the list
print(vals[::-1])     # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]

# Start at 1, stop at 8, skip by 3
print(vals[1:8:3])    # [1, 4, 7]
```

| Adding Elements to Lists

.append(x)

Adds x to the end of the list.

```
[1, 2].append(3) → [1, 2, 3]
```

.insert(i, x)

Adds x at position i.

```
[1, 3].insert(1, 2) → [1, 2, 3]
```

.extend(list)

Appends items from another list.

```
[1].extend([2, 3]) → [1, 2, 3]
```

| Removing List Elements

```
tasks = ["code", "eat", "sleep", "repeat"]

# Remove by value
tasks.remove("eat")    # ["code", "sleep", "repeat"]

# Remove and return by index
done = tasks.pop(1)    # done = "sleep", tasks = ["code", "repeat"]

# Clear everything
tasks.clear()          # tasks = []
```

⚠️ Pro Tip: `pop()` without an index removes the last item!

| Sorting & Reversing

In-Place vs. New List

```
scores = [85, 92, 78, 90]

# Modifies 'scores' directly
scores.sort()

# Returns a new list, leaves
# 'scores' original
new_list = sorted(scores)
```

Strings Sorting

Strings are sorted lexicographically (A-Z, then a-z).

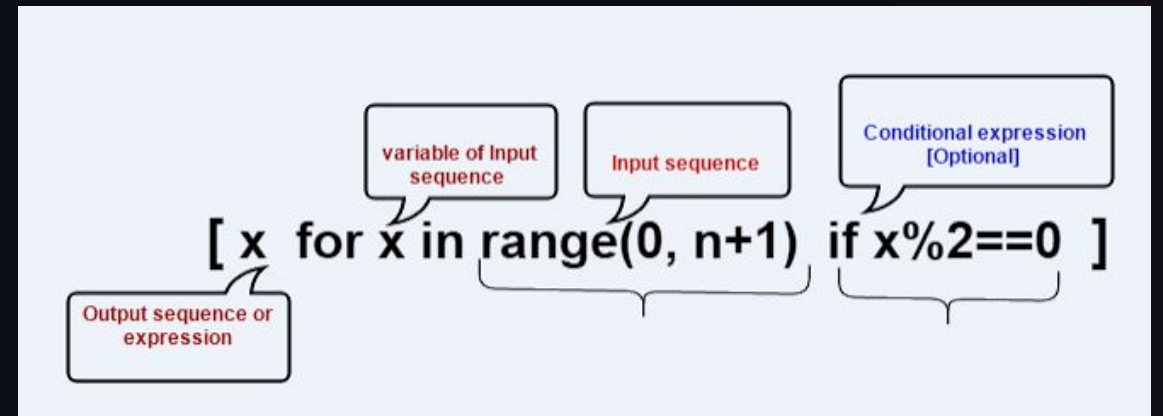
```
["Z", "a"].sort() → ["Z", "a"]
```

| List Comprehensions

The "Pythonic" way to create lists from existing iterables.

```
# Traditional Loop
sq = []
for n in range(5):
    sq.append(n**2)

# Comprehension
sq = [n**2 for n in range(5)]
```



| Filtered Comprehensions

Add logic inside the brackets to filter items.

```
# [expression for item in list if condition]

nums = [1, 2, 3, 4, 5, 6]

# Keep only even numbers
evens = [n for n in nums if n % 2 == 0]

# Square even numbers
sq_evens = [n**2 for n in nums if n % 2 == 0]
```

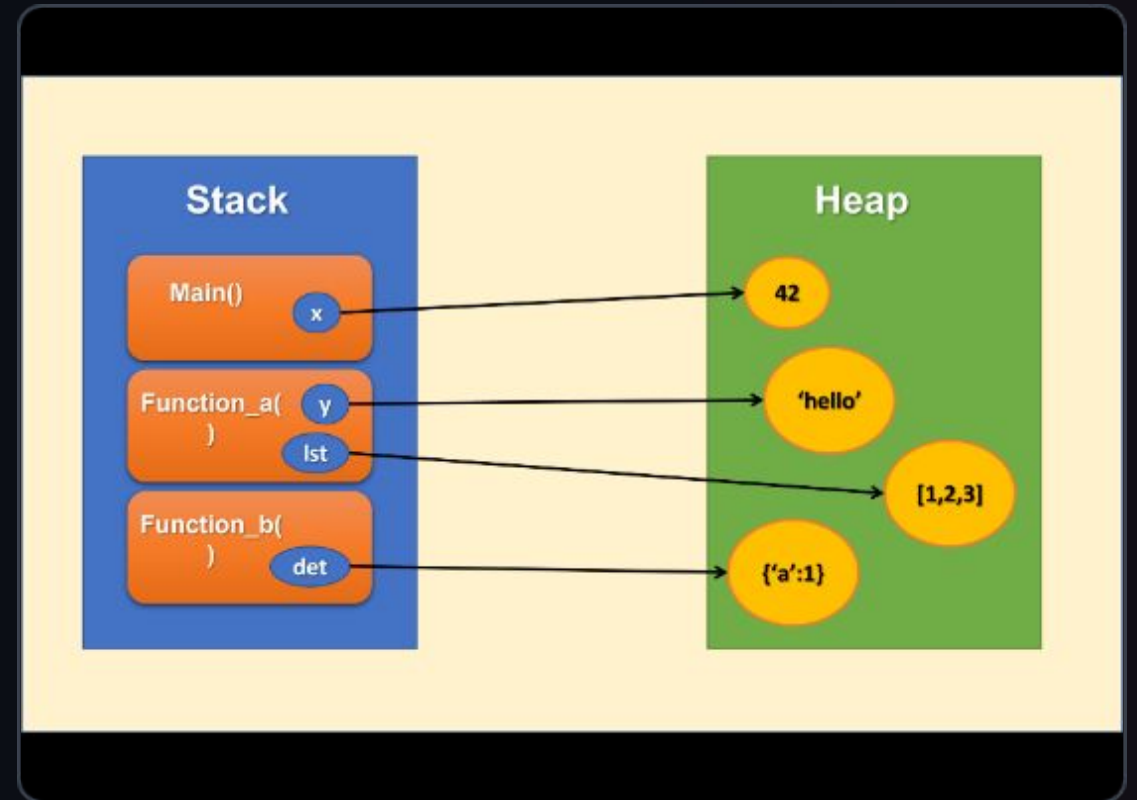
| Lists: References in Memory

Variables store **references** to objects, not the values themselves.

```
a = [1, 2, 3]
b = a # Both point to SAME list!

b.append(4)
print(a) # Output: [1, 2, 3, 4]
```

To copy a list truly, use `c = a.copy()` or `c = a[:]`.



| Practice: List Challenge

Task: Extract and Sort

Given the list data = [10, 5, 20, 15, 30]:

1. Get the first 3 elements.
2. Add 50 to the end.
3. Sort the entire list in descending order.
4. Print the result.

```
# Solution Preview  
result = sorted(data[:3] + [50], reverse=True)
```

| Tuples: Immutable Safety

Tuples are **ordered** but **immutable** (cannot be changed after creation).

```
# Tuple Syntax (Parentheses)
coords = (40.7128, 74.0060)

# This will fail:
coords[0] = 50.0
# TypeError!
```

Use Cases

- Constant records
- Dictionary keys
- Function return values
- Performance-critical data

| Packing & Unpacking

Packing

Assigning multiple values to one variable.

```
point = 3, 4, 5
```

Unpacking

Extracting values back into variables.

```
x, y, z = point
```

The Pythonic Swap:

```
a, b = b, a
```

| The Asterisk (*) Operator

Use * to capture the "rest" of the items into a list.

```
stats = (10, 20, 30, 40, 50)

# Unpack first and rest
first, *others = stats
# first = 10, others = [20, 30, 40, 50]

# Unpack first, middle, and last
first, *mid, last = stats
# mid = [20, 30, 40]
```

| Lists vs. Tuples

Feature	List	Tuple
Mutability	Mutable (can change)	Immutable (fixed)
Syntax	[]	()
Memory	Higher overhead	Leaner/Faster
Methods	Many (.append, .sort...)	Few (.count, .index)

| Session 2: Dictionaries

Dictionaries store data in **Key-Value** pairs. They are incredibly fast for lookups.

```
# Syntax: {key: value}
user = {
    "id": 101,
    "name": "Alice",
    "email": "a@mit.edu"
}
```

Dictionary in Python

Dict = {1: "Python", 2: "Java", 3: "C++"}



| Dictionary Key Rules



Keys: Immutable

Keys must be **hashable**. Use strings, numbers, or tuples. Never use lists.



Keys: Unique

If you add a duplicate key, the old value is overwritten.



Values: Any type

Values can be anything—even other dictionaries!

| Safe Data Retrieval

```
hero = {"name": "Batman", "city": "Gotham"}
```

Method 1: Brackets

```
print(hero["name"]) # "Batman"
```

Method 2: .get() (Preferred)

Returns None instead of crashing if key is missing

```
print(hero.get("power")) # None
```

.get() with default value

```
print(hero.get("power", "Human Strength")) # Prints Human Strength
```

```
data = {
    'name': 'Felix',
    'age': 20
}

height = data['height']
```



```
KeyError
Cell In[9], line 6
      1 data = {
      2     'name': 'Felix',
      3     'age': 20
      4 }
----> 6 height = data['height']

KeyError: 'height'
```

```
data = {
    'name': 'Felix',
    'age': 20
}
```



```
height = data.get('height', None)
```

✓ 0.0s

```
def node (state:State) -> State:
    "A custom LangGraph Node"

    msg = state.get('messages', [])
    summary = state.get('summary', '')

    "Rest of the code for the node"
```



| Adding & Deleting Items

```
car = {"brand": "Tesla", "year": 2020}

# Update or Add

car["year"] = 2023 # Updates value

car["model"] = "S" # Adds new pair

# Delete
del car["brand"] # Permanent removal
val = car.pop("year") # Removes and returns 2023
```

Adding & Updating Items

Add new key:

```
python Python
student["grade"] = "A"
```

Update value:

```
python Python
student["age"] = 22
```

- Dictionaries update **instantly**.

Removing Items

```
python Python
student.pop("grade") # removes key
student.popitem()    # removes last item
del student["age"]   # delete specific key
student.clear()       # empty dictionary
```

| Iterating over Dictionaries

Key Patterns:

```
# 1. Iterate over items (Pairs)
for k, v in my_dict.items():
    print(f"{k} is {v}")

# 2. Iterate over keys
for key in my_dict.keys(): ...

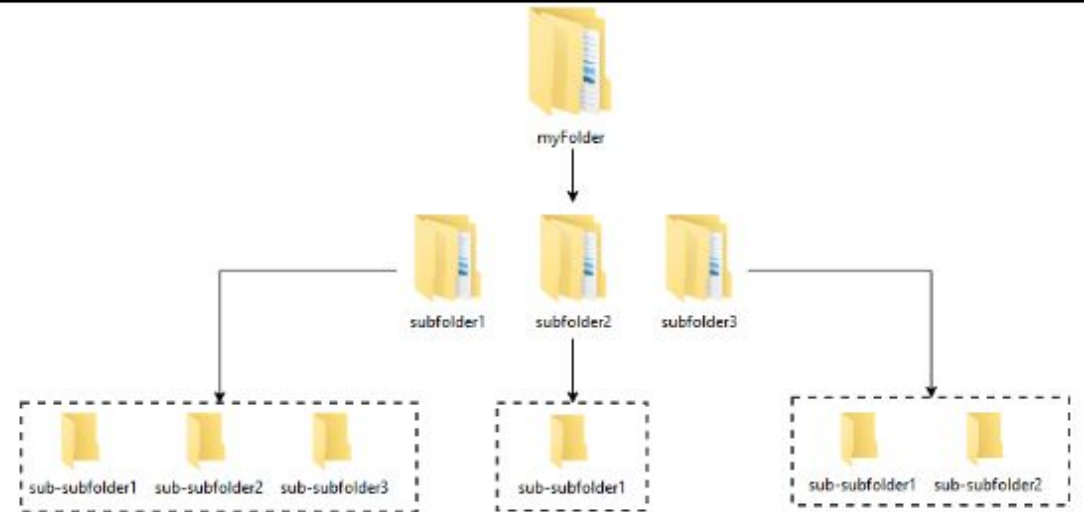
# 3. Iterate over values
for val in my_dict.values(): ...
```

🕒 Note: Since Python 3.7+, dictionaries preserve the **insertion order** of items.

| Nested Dictionaries

```
class_2024 = {  
    "alice": {"gpa": 3.9, "major": "CS"},  
    "bob":   {"gpa": 3.7, "major": "EE"}  
}
```

```
# Accessing nested data:  
major = class_2024["alice"]["major"]
```

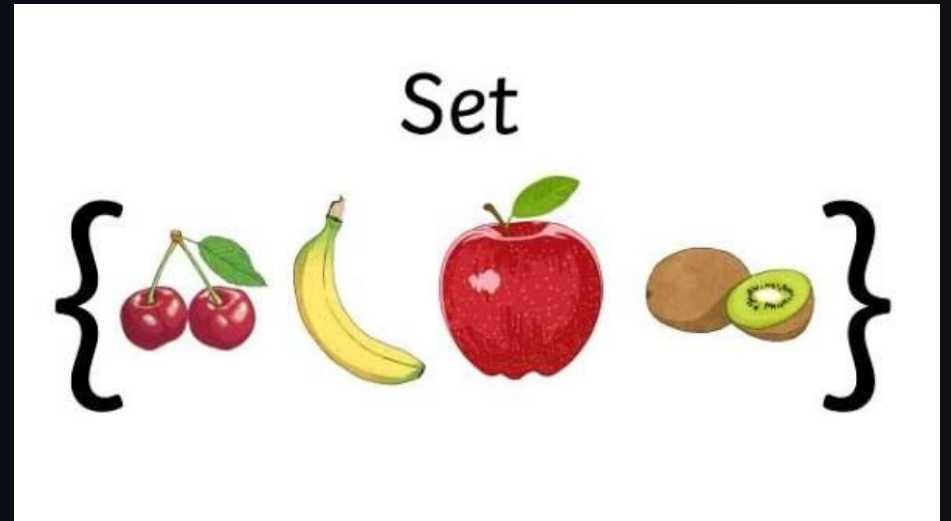


| Sets: Unique Collections

Sets are **unordered** collections of **unique** items.

```
# Syntax: {val1, val2}
ids = {10, 20, 10, 30}
print(ids) # {10, 20, 30}

# Membership test (Very Fast!)
if 10 in ids:
    print("Found!")
```



| Set Math Operations

```
A = {1, 2, 3}
```

```
B = {3, 4, 5}
```

```
# Union (All items)
```

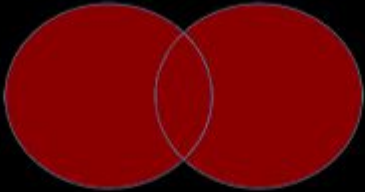
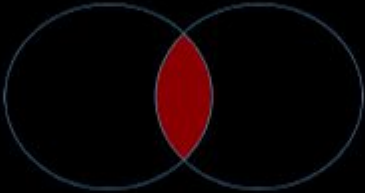
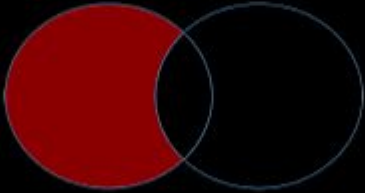
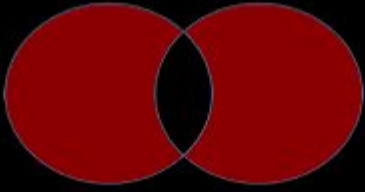
```
A | B # {1, 2, 3, 4, 5}
```

```
# Intersection (Common)
```

```
A & B # {3}
```

```
# Difference (In A not B)
```

```
A - B # {1, 2}
```

Set Operation	Venn Diagram	Interpretation
Union		$A \cup B$, is the set of all values that are a member of A , or B , or both.
Intersection		$A \cap B$, is the set of all values that are members of both A and B .
Difference		$A \setminus B$, is the set of all values of A that are not members of B
Symmetric Difference		$A \triangle B$, is the set of all values which are in one of the sets, but not both.

| Performance Benchmark

Finding an item (x in collection):

Structure	Complexity	Analogy
List	$O(n)$ - Linear	Checking every page of a book
Tuple	$O(n)$ - Linear	Checking every page of a book
Set	$O(1)$ - Constant	Using an index to find the page
Dict	$O(1)$ - Constant	Using an index to find the page

| Choosing the Structure

☰ **List:** Ordered, changeable, duplicates okay.

🔒 **Tuple:** Ordered, safe, constant data.

📖 **Dict:** Key-value mapping, ultra-fast lookup.

💎 **Set:** Unique items, math logic.

🏆 Summary Table

Data Structure	Ordered?	Mutable?	Allows Duplicates?	Best For
List (<code>list</code>)	✅ Yes	✅ Yes	✅ Yes	Dynamic collections, maintaining order
Tuple (<code>tuple</code>)	✅ Yes	❌ No	✅ Yes	Fixed collections that shouldn't change
Dictionary (<code>dict</code>)	❌ No	✅ Yes	❌ No (keys must be unique)	Fast lookups, key-value storage
Set (<code>set</code>)	❌ No	✅ Yes	❌ No	Unique collections, removing duplicates

| Choosing the Structure

Benefits of Choosing the Right Data Structure



Improved Efficiency

Faster data access and manipulation.



Enhanced Clarity

Cleaner, more maintainable code.



Reduced Usage

Optimal resource utilization.



Enabled Scalability

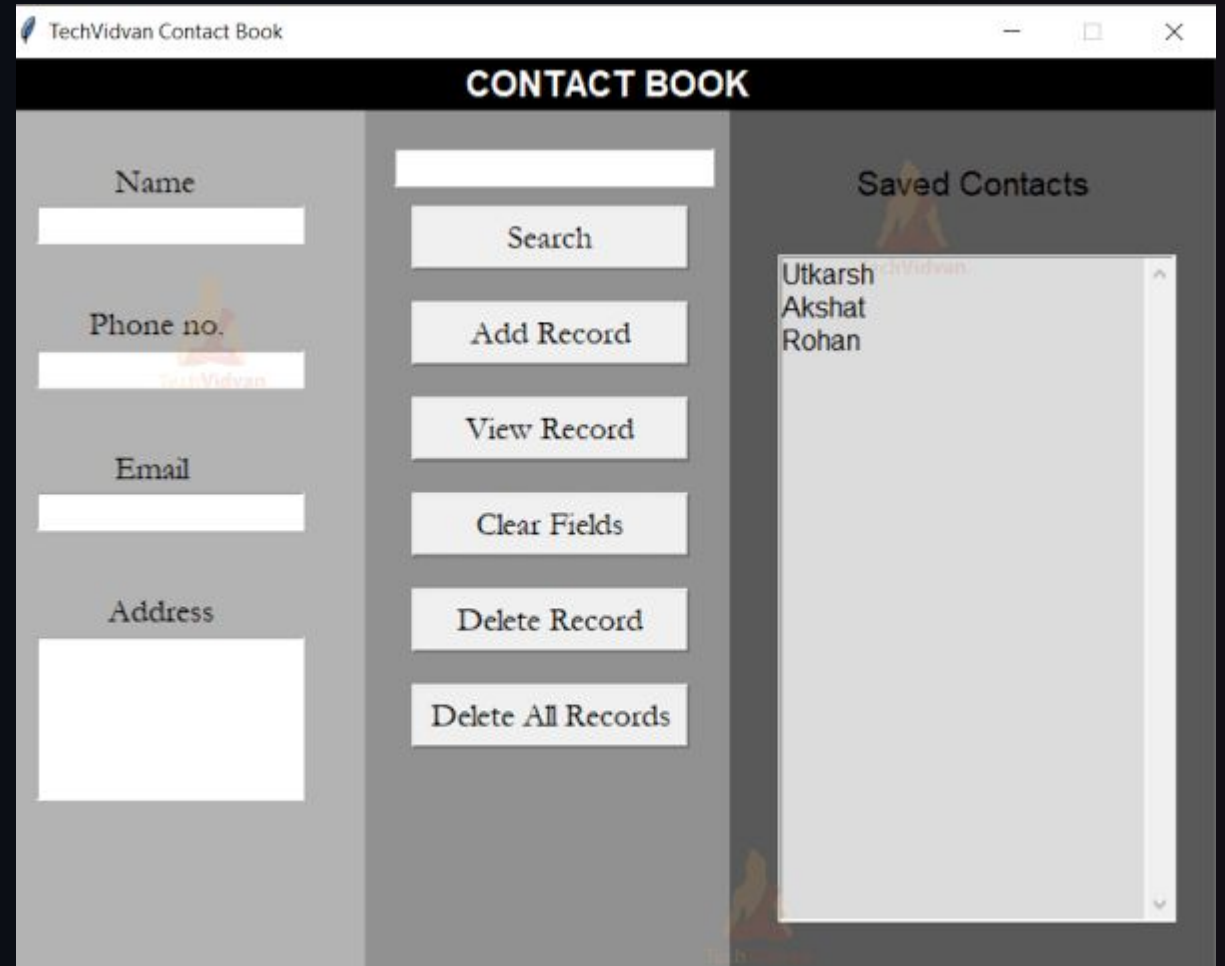
Handles growth without major refactoring.

| Weekend Project

The Contact Book Manager

Build a program that stores personal contacts using a **Dictionary**.

- Store names as keys.
- Store nested dicts (Phone, Email) as values.
- Implement Search, Add, and Delete features.
- **Bonus:** Prevent duplicate emails using a Set.



The screenshot shows a web application titled "TechVidvan Contact Book". The interface is divided into three main sections:

- Left Section (Form):** Contains input fields for "Name", "Phone no.", "Email", and "Address".
- Middle Section (Buttons):** Contains a search bar, a "Search" button, and five action buttons: "Add Record", "View Record", "Clear Fields", "Delete Record", and "Delete All Records".
- Right Section (Saved Contacts):** Titled "Saved Contacts", it displays a list of names: "Utkarsh", "Akshat", and "Rohan".

The application has a dark theme with white text and buttons. A small "TechVidvan" logo is visible in the bottom right corner of the interface.

Week 2 Complete

02

Next Week: Functions, File I/O, & Exceptions

Go forth and collect data!

♣ python.org/docs